

## Unit-I

### ➤ Introduction to Database Management System (DBMS)

**Database:** “A database is an organized collection of large amounts of structured information, or data stored in a computer system in tabular format.”

- The data or information stored in databases can be easily accessed, managed, modified (updated), controlled, and organized, for that most databases uses structured query language (SQL).

For example, a **company** database may include different tables like Products, Employee, and financial details. Each of these tables would have different fields that are relevant to the information stored in the table which is shown in following table.

| Database: Company |           |               |             |       | Table 2) Employee |          |             |           | Table 3) Finance Details |           |            |        |
|-------------------|-----------|---------------|-------------|-------|-------------------|----------|-------------|-----------|--------------------------|-----------|------------|--------|
| Table 1) Product  |           |               |             |       |                   |          |             |           |                          |           |            |        |
| Prod ID           | Prod Name | Date of Prod. | Exp. Date   | Price | Emp Id            | Emp Name | Designation | Basic Pay | Fin Id                   | Aca. Year | Fin Detail | Amt    |
| P1                | abc       | 1-Feb-2000    | 1-Mar-2000  | 55    | E1                | Ajay     | Manager     | 9000      | F1                       | 1999      | Profit     | 5 Lakh |
| P2                | xyz       | 4-Jun-2000    | 4-Aug-2000  | 78    | E2                | Mahesh   | Head Clerk  | 6000      | F2                       | 2000      | Loss       | 2 Lakh |
| P3                | pqr       | 7-Aug-2000    | 7-Dec-2000  | 75    | E3                | Kartik   | Jr Clerk    | 5000      | F3                       | 2001      | Profit     | 3 Lakh |
| P4                | mno       | 15-Dec-2000   | 15-Mar-2001 | 80    | E4                | Pawan    | Peon        | 3000      | F4                       | 2002      | Profit     | 1 Lakh |

- We use Facebook, it has its own database to store information of users and their activities.
- Also, we use Flipkart, amazon, Snapdeal etc. e-commerce websites and all these websites have their own separate databases that stores information about products, customers, transactions etc.

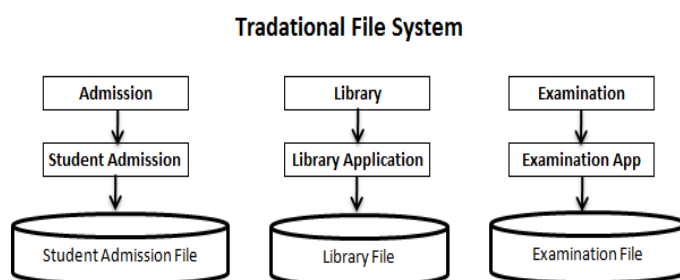
#### Features of Database:

Following are the several key features of database:

- In database, all data or information stored at one **central** location in tabular format.
- Database allows data to be shared by many users at one time.
- Database provides user interface to work with the data.
- Database creates backups (Extra copy) for data after specific time interval.
- Database controls who can access and edit the data by defining roles for user.

#### Traditional File system:

- Before the database, a manual file system was used to maintain the records and files. Data were stored and processed using a traditional way and it took much more time for processing.
- In traditional file system, all data is stored in different files and each file is independent of other file. All data in the different files can be integrated only by writing an individual program for each application.
- A traditional file system uses File system. A **file system** is an application or software used to store and organize a set of computer files. It maintains the physical location of the files on storage device.
- Diagram shows traditional file system consisting different independent files VIZ, student Admission file, Library File and examination file which are managed by Student App, Library App and Examination App.



## Limitations/Drawback/Disadvantages of traditional file processing system:

### 1) Data redundancy:

- In traditional file system, there is a data redundancy (duplication of data i.e. multiple copies of the same data.) it means there is repetition of data is found.
- We know that, file system has multiple application programs, each application program has its own data file, all files are independent to each other and hence these files may contains duplicate data that creates 'data redundancy'. This duplication of data creates many problems such as it requires additional storage space, time, money and requires additional efforts to keep all files up to-date.

**Example:** A student is having record in admission file and library file. The Name, roll number and class will be same in both files. Also, these files are independent to each other. So, it creates lots of duplication of student data.

### 2) Data-inconsistency:

- Data inconsistency occurs due to the same data is stored in multiple files (i.e. because of data redundancy), if some data to be changed then all the files containing that data needs to be modified.
- Data inconsistency means that different files may contain different information of a particular object because these files are independent.

**Example:** If you change student name in library file then name should be changed in admission file also. If name isn't changed then it may create a risk of data inconsistency.

### 3) Difficulty in accessing data:

- In a file processing system, it is difficult to access or retrieve specific type of data from all files.
- To access or retrieve specific data from different files, then programmers have to create separate applications.
- Suitable or specific information access is almost impossible in file processing system that leads for difficulty to access the data.

**Example:** If college admin wants student information like his name, roll number, class, marks and percentage then program is written already, but if he wants those students records who secured more than 80 percent. Then there is need to create or develop a separate application that produce result as per college admin.

### 4) Data-isolation:

- We know that- in file processing system, data is scattered or stored in various files and these files may be in different formats and they are independent to each other.
- If we want to extract data from two or more files, then it is very difficult to write new application program to retrieve the appropriate data that leads for data isolation.

**Example:** In Bank System,

- Saving Account file has address fields: Add line1, City, State, Pin code.
- Current account file has address fields: House No, Street No, City, State, Pin code.

Both these files have same fields like city, state, Pin code. If Bank Manager wants list of customers who living in a particular city, then he retrieves list of the entire customer's data from both files this occurs due to data was stored in different files having different format.

### 5) Data Atomicity:

- Data atomicity, is a core concept in database transactions, that ensures all transactions are completed successfully, or none of them. Data Atomicity gives guarantee that data consistency and preventing partial updates.
- Atomicity saves such transactions only which are either completed (100%) or that are canceled (0%) i.e. partial updation is avoided. If system failed at any time, the data should be restored to the consistent state that existed before to the failure.

- It is difficult to maintain atomicity in file processing system due to data redundancy, data isolation etc.

**Example:** Suppose a user wants to transfer money from **account A** to **account B**. This process consists of two steps:

- Deduct the money from **account A**.
- Add the money to **account B**.

If a system failure occurs during the execution of fund transfer operation, it may be possible that, amount is debited from **account A**, but it is not credited to **account B**. This situation can make data inconsistent and data atomicity avoids such incomplete transaction such that system remain consistent as before the failure of transaction.

## 6) Poor Data Security:

- In file processing system, there is no security provided for data stored in file. And hence, the unauthorized user can easily access, modify and add the data stored in the files.

**Example:** A student can easily access his data in the college library file, then he can easily change his book issue details such as submission of book, change his fine to zero etc. because there is no security for data.

## 7) Poor Data Integrity:

- Integrity means reliability (trustworthiness) and accuracy of data. The data values stored in the database must satisfy certain types of constraints (Rule). While inserting the data into database these constraints must be validated. If data satisfies all constraint, then only it will store in databases.
- But, in file system there is no checking of such constraints. Therefore, wrong data may be inserted into file system. Because, it is very difficult to apply constraints on files and it leads for poor data integrity.

**Example:** While inserting phone number into database we apply constraint like- it cannot be greater than 10 digits. Such constraints are applicable in databases not in file system.

## 8) Data Sharing:

- We know that, in file system data is scattered in various files with different formats. Due to this sharing of data among different applications is hard in file system.
- The file-based system does not provide data sharing among multiple users on the network.

## 9) Concurrent access anomalies (irregularities):

- If multiple users are updating the same data simultaneously, it will result in inconsistent data state.
- In file processing system, it is very difficult to handle simultaneous updation of data and that leads for concurrent access anomalies.

**Example:** Bank **account A** has balance Rs.500. If two customers withdraw funds Rs.50 and Rs.100 respectively from **account A** at the same time, then the result of the concurrent executions may lead the account in an incorrect (inconsistent) state.

## 10) Data Dependence:

- In file-based processing systems, structure of data files is attached with application program. If format of file is changed, then its respective application needs to be changed. That is, there is dependency between file structure and its respective application. To maintain such dependency in file system is tough task.

## 11) Backup and Recovery:

- In traditional file system, a user needs to take backup of data after a regular interval of time that wastes lots of time and resources. If the volume of data is large then this process may take a very long time. Also, data recovery in file system is hard to implement.

**DBMS:** The Database Management System (DBMS) is a software or application designed or developed to store, retrieve, update, and manage the data stored in a database.

Following are some examples of DBMS.

E.g.- Microsoft Access, Oracle, MySQL, SQL Server, PostgreSQL, FileMaker, RDBMS, dBASE, Clipper, FoxPro etc.

Following Diagram shows DBMS:



In above diagram, user can store, access, update and manage the data stored in database using different application programs with the help of DBMS via structured query language (SQL)

### **Advantages of Database or advantages of DBMS:**

DBMS offers numerous advantages for data management, which are as follows-

#### **1) Minimize Data Redundancy:**

- We know that, in File Processing System, same data is stored in different files (duplication of data) that makes data redundancy which leads to waste in memory space, time, money and efforts.
- But data redundancy is reduced or eliminated or minimized in DBMS because all the data is stored in a single database (at centralized location) hence there is no chance of duplicate data. Thus, data redundancy is minimized in DBMS.
- **Example:** A student record in library or examination can contain duplicate values, but when they are converted into a single database, all the duplicate values are removed.

#### **2) Data Consistency:**

- We know that, in DBMS all data is stored in a single database (at centralized location) in tabular format. And all tables are inter-related with each other. Changes made in one table that also affects related data values in other table hence we will get consistence data at all time.
- Thus, Data consistency allows same or similar data values for every user whenever necessary.

#### **3) Sharing Of Data:**

- We know that, in DBMS all data is stored at centralized place therefore it can easily be shared by number of authorized users. The database administrator manages the data and gives rights to users to access the data.
- The remote users (users over the network) can also share same data stored in database. Similarly, the data of same database can be shared between different application programs.

#### **4) Data integration:**

- We know that, in DBMS, Data is stored in different tables all these tables are inter linked with each other and many users feed entries in these tables so it is important to maintain association between data items and DBMS allows such data association allowing user to manage data items concurrently and makes data integration easily.

## 5) Data Security:

- Data security means protecting a data from unauthorized access. Data in database should be kept secure and safe from unauthorized access allowing user to prompt for user name and password. Only those users are authorized can manage the data stored in databases.
- DBMS always keep database secure and theft free.

## 6) Data Integrity:

- Integrity means reliability (trustworthiness) and accuracy of data. The data values stored in the database must satisfy certain types of constraints (Rule). While inserting the data into database these constraints must be validated. If data satisfies all constraint, then only it will store in databases.
- The DBMS allow user to enter only valid data in database with the help of constraints.
- User enters data to store in database, DBMS first check entered data is correct or not using constraints. If it founds wrong data then data is not stored in database.
- The constraints may be applied on single or multiple columns of table.
- DBMS maintain the integrity of data, provide capabilities for defining and enforcing the constraints.

**Example:** If user wants to store salary of an employee within the range 10000 to 40000, Then using CHECK (salary>=10000 AND salary<=40000) constraint it is validated. If entered salary amount is within range specified, then only it will store in database otherwise not.

## 7) Data Atomicity:

- Data atomicity, is a core concept in database transactions, that ensures all transactions are completed successfully, or none of them. Data Atomicity gives guarantee that data consistency and preventing partial updates.
- Atomicity saves such transactions only which are either completed (100%) or that are canceled (0%) i.e. partial updation is avoided. If system failed at any time, the data should be restored to the consistent state that existed before to the failure.
- DBMS doesn't allow the system to be in a partially committed state.

**Example:** In railway reservation system, if user has completed the process of ticket reservation, then his record will be stored and amount of ticket will be deducted from his account otherwise no amount will be deducted. If ticket is not booked and amount is deducted then it will be rolled back.

## 8) Concurrency Control:

- DBMS provide access to the database for multiple users at the same time and this creates concurrency.
- DBMS has the power to control concurrency, so that no transactions are lost and maintain data consistency.
- DBMS has its own mechanism to handle concurrency accesses and avoids any loss of information.

## 9) Data Independence:

- The separation of data from its application program is known as data independence.
- In DBMS, database and application program are maintained separately from each other. DBMS acting as a mediator between them.
- You can easily change the database structure without affecting its respective application program and it's leads for data independency.

## 10) Backup and Recovery:

- DBMS have built-in backup-and-recovery facility that automatically takes backs-up of all important data, and restores it whenever necessary.

**Example:** If a system fails in the middle of any process, then DBMS stores the values of that state in which database was before query execution.

## Disadvantages of DBMS:

- 1) **Costly:** DBMS requires high initial investment for hardware, software and trained staff hence it is costly.
- 2) **Complexity:** A DBMS fulfill lots of requirement and solves many problems of database. But all these functionalities have made DBMS extremely complex software.
- 3) **Requirement of Technical staff:** If any problem occurs in DBMS, then technical staff is required, who have knowledge to understand & solve the DBMS problems.
- 4) **Loss of Data:** We know that, In DBMS, all the data is stored in single database, so if database or any component of database fails then it leads for loss of valuable data.
- 5) **Cost of Data Conversion:** In some situation, we need to convert the data from one database to another database, this requires a lot of money for data conversion, because for this conversion we need to hire database system designers whom we have to pay a lots of money.
- 6) **Currency Maintenance:** We know that, the currency values are day by day changes and DBMS requires to updates itself daily if there is currency field in database.
- 7) **Requires lots of Memory space:** We know that, DBMS is big and complex software due to its operations or functionalities, so it requires lots of memory space to run efficiently.

## Difference between DBMS and File System:

Following table shows difference between DMBS and File system:

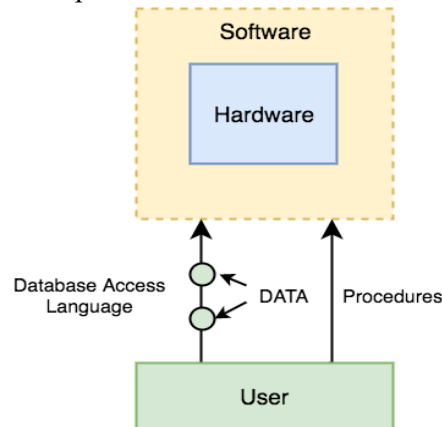
| DBMS                                                                                                                                                                                                            | File System                                                                                                                                                                                    |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1) In DBMS, there is <u>minimum data redundancy</u> . Since all data is stored at central location.                                                                                                             | 1) In File system, there is <u>maximum data redundancy</u> . Since all data is stored in different-different files.                                                                            |
| 2) <u>Data consistency is well managed in DBMS</u> . Since, all data is stored in tables and these tables are inter liked with each other. Hence, changes made in one table that also reflect in another table. | 2) In file system, there is <u>data inconsistency</u> because of data is scattered in different files. If changes made in one file that not reflect in another file causes data inconsistency. |
| 3) In DBMS, all data is stored at one location therefore data in DBMS can easily be shared by multiple users.                                                                                                   | 3) In file system, all data is scattered in different files therefore sharing of data in file system in hard.                                                                                  |
| 4) DBMS have built in functionality to handle the concurrency transactions such that data consistency will be maintained.                                                                                       | 4) File system does <b>not</b> have functionality to handle the concurrency therefore there will be data inconsistency.                                                                        |
| 5) DBMS provide high security for data by prompting user name and password. Therefore, only authorized user can access the data.                                                                                | 5) File system does <b>not</b> provide security for data. Therefore, unauthorized user can also access the data.                                                                               |
| 6) DBMS have functionality to take automatic backup of data after specific time interval. Meanwhile if any problem occurs then system can easily be recovered from backup files.                                | 6) File system does <b>not</b> provide back up and data recovery. If system failure occurs then all the data may be lost.                                                                      |
| 7) DBMS ensures that data stored in database in correct by validating data values using constraints.                                                                                                            | 7) File system unable to validate data values using constraints hence wrong data may be inserted into file.                                                                                    |
| 8) In DBMS, database and application program are maintained separately and DBMS acts as a mediator between them. So, data stored in database and application program are independent of each other.             | 8) In file system, there is data dependency between application program and its respective file. If file structure changed then its respective application program needs to be update.         |

# Components of DBMS:

A Database Management System (DBMS) is composed of several key components, which are as follows-

1. Hardware
2. Software
3. Data
4. Procedures
5. Database Access Language
6. User

Following diagram shows database components:



## 1. Hardware:

- We know that, Hardware is nothing but any physical part or components of computer system, that includes hard disks, I/O channels, CPU, memory, keyboard, mouse etc.
- Such hardware components are necessary to run DBMS because user enters commands via keyboard, send the data via I/O channels, process that data via CPU and it will then get stored in memory (RAM/ROM).

## 2. Software:

- DBMS software is the main component of DBMS, as this is the program which controls everything in database.
- The DBMS software provides an easy-to-use interface for user to store, access and update the data of database.
- Also, the DBMS software is capable of understanding the Database Access Language and interpret it into actual database commands to execute them on the database.

## 3. Data:

- Data is that resource, for which DBMS was designed. The main idea or motive behind the creation of DBMS is to store and utilize data.
- We know, that all user's data is stored in database which is managed by DBMS.
- When data is stored in database then DBMS also store its data about the data (Metadata).
- This metadata is used for better understanding of the stored data.
- For example: When we store Name in a database, then DBMS will store extra information related with that name like when the name was stored in the database, what is the size of the name, is that stored name is related with some other data, or is it independent, all this information is metadata.

## 4. Procedures:

- Procedure refers to set of instructions or statements which are developed or created to manage the DBMS.
- These procedures include setup and install a DBMS, to login and logout of DBMS, to manage databases, to take backups, generating reports etc.

## 5. Database Access Language:

- We know that, languages are used for interactions. Here, database languages are used to interact with DBMS.
- Database Access Language is designed to write commands to access, insert, update and delete the data stored in any database.
- E.g. **SQL** (Structured Query Language) is the database access language.

## 6. User:

- A user is a person who can use and take the benefits of database.
- User write commands using Database Access Language then submit it to the DBMS for execution, which are then translated and executed by the DBMS.
- Also, user can create new databases, tables, insert data, fetch stored data, update data and delete the data using the database access language.

## Database users: (DBMS users)

- Database user is person who use and take the benefits of database.
- Depending upon user interaction with database, there are several types of DBMS users which are as follow-
  - 1) Database Administrator (DBA)
  - 2) Parametric End user (Naive user)
  - 3) System Analyst
  - 4) Sophisticated users
  - 5) Database Designers
  - 6) Application programmer
  - 7) Casual users (Temporary users)

Let's see these users in details:

### 1) Database Administrator (DBA):

- Database Administrator (DBA) is a user account in DBMS who defines the schema (a schema refers to the logical design or blueprint of the entire database) and also controls the three levels (Physical, logical and view level) of database.
- The DBA has rights to create a new user account by assigning user id and password.
- DBA is also responsible for providing security to the database and it allows only the authorized users to access or modify the data stored in database.
- DBA also monitors the back up and takes recovery whenever necessary and provide technical support
- DBA repairs database damage caused due to hardware or software failures.
- The DBA has a DBA account in the DBMS which is called a "system or super user account". i.e. "system" is DBA user account in DBMS.

### 2) Parametric End Users: (Naive users)

- Parametric End Users (naive users) are the unsophisticated (unexperienced) users who don't have any DBMS knowledge but they frequently use the database in their daily life.
- These user uses the pre-developed applications to perform their tasks on database.
- For example, Clerks in any bank is a naive user because they don't have DBMS knowledge but they still use the database and perform their given task with the help of banking software (Pre developed Application).

### 3) System Analyst:

- System Analyst is a database user who analyzes the requirements of parametric end users.
- They check whether all the requirements of end users are fulfilled or not.
- System Analysts come in the middle of end users and the technical support staff. They understand the requirements of the end users and translate them into technical needs and handed over these needs towards technical staff (Database designer or programmer).
- Example: A system analyst working on an online learning platform needs to make a list of requirements from the teachers and students. And this requirement list then handed over to technical staff.

### 4) Sophisticated Users:

- Sophisticated users can be database engineers, scientists, database analyst, who are familiar with the database.

- They can develop their own database applications according to their requirement.
- They don't write any program code but they interact with the database by writing SQL queries directly through the query processor or by SQL command line interface.

### 5) Database Designers:

- Data Base Designers are the users who design the structure of database that includes tables, indexes, views, constraints, triggers, stored procedures.
- These user controls what data must be stored in database? and how the data items to be related with each other?

### 6) Application Programmer:

- Application Programmer are the back-end programmers who writes the code for the application program that interact with database.
- They are the computer professionals i.e. software developers.
- These programs could be written in Programming languages such as C, C++, Java, C#, Python etc.

### 7) Casual Users (Temporary Users):

- Casual Users are the users who occasionally use or access the database but each time when they access the database, they require the new information
- for example, Middle or higher-level managers such as principal of college, manager of bank, CEO (Chief Executive Officer) of company etc. are the casual user.

## Roles of DBA (Functionalities of DBA):

The database administrator's responsibilities can include the following tasks:

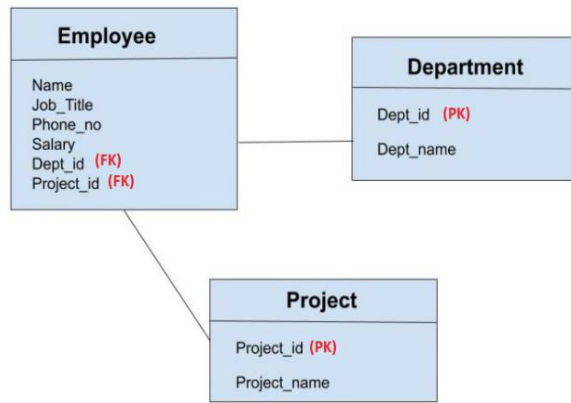
- Installing and upgrading the database server and its application tools.
- Allocating storage for the database system and planning future requirements.
- Modifying the database structure and ensuring key constraints whenever necessary.
- Granting User Access and maintaining system security.
- Controlling and monitoring user access to the database.
- Monitoring performance of database.
- Ensuring fulfillment of database vendor license agreement.
- Contacting database vendor (seller) for technical support.
- Take time to time Backup and recovery of databases whenever necessary.

## Database structure:

- We know that, a database is an organized collection of large amounts of structured information or data stored in a computer system in table format.
- Within a database, related data are grouped into tables, each of which consists of rows (also called tuples) and columns, like a spreadsheet.
- To convert your lists of data into tables, start by creating a table for each type of entity, such as products, sales, customers, and orders.
- Each row of a table is called a record or Tuple. Records include data about something or someone, such as a particular customer information.
- By contrast, columns (also known as fields or attributes) contain a single type of information that appears in each record, such as the customers information is listed in following table.

| First Name | Last Name | Age | PIN Code |
|------------|-----------|-----|----------|
| Ramesh     | Mane      | 43  | 413307   |
| Kiran      | Patil     | 32  | 541120   |
| Samir      | Khan      | 56  | 425506   |

- In database all the information is stored in different-different tables. And these tables are linked with each other via. Primary key, foreign key or reference key constraints shows in following diagram.



To keep the data consistent in table, we have to assign the appropriate data type to each column or attribute.

Common data types include:

CHAR - a specific length of text

VARCHAR - text of variable lengths

TEXT - large amounts of text

INT - positive or negative whole number

FLOAT, DOUBLE - can also store floating point numbers

BLOB - binary data

## DBMS Architecture:

- The main purpose of Database Architecture **is to store and manage data in a proper way.**
- The design of a DBMS depends on its architecture. DBMS architecture helps to design, develop, implement, and maintain the database.
- The design of DBMS is in three forms: centralized (all the data stored at one location), decentralized (multiple copies of database stored at different locations) or hierarchical (Tree structure).

**Database Architecture are of three types: -**

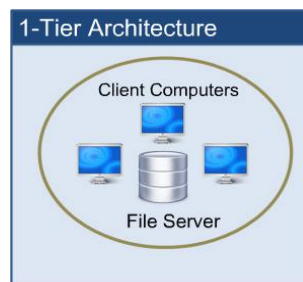
- 1-tier architecture
- 2-tier architecture
- 3-tier architecture

Let's see all architectures in details:

### 1-tier architecture:

- In 1-tier architecture, **the database is directly available to the user** for accessing and storing data.
- Generally, such setup is **used for local application development, where programmers directly communicate with the database** for quick response.
- In this 1 tier architecture, **the Client, Server and Database all resides on the same machine.**
- In 1-tier architecture, anytime we can access database easily.
- **Database designers and programmers normally use a single-tier architecture.**
- It is very simple and easy to use. But such architecture is rarely used in production.

Following diagram shows 1-tier architecture:

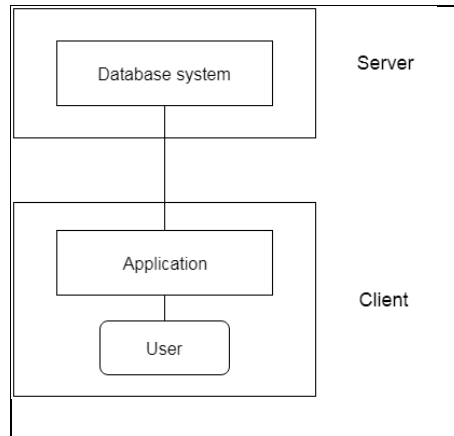


**Example:** Suppose we want to access the records of employee from the database and the database is available on same computer, so the request done by you and the employee records are accessed from the database. This type of system is local database system and termed as 1-tier architecture.

## 2-tier architecture:

- 2-tier architecture is developed by using **Client-Server architecture**.
- In the two-tier architecture, **applications on the client end can directly communicate with the database at the server side**. For this interaction, API's like: ODBC, JDBC are used.
- The server side is **responsible to provide the functionalities requested by the clients using query processing and transaction management**.
- In this architecture, to communicate with the DBMS, **client-side application establishes a connection with the server**.
- 2 tier architecture provides **security to the DBMS** as it is not exposed to the end user directly.
- This architecture provides Direct and faster communication.

Following diagram shows 2-tier architecture:

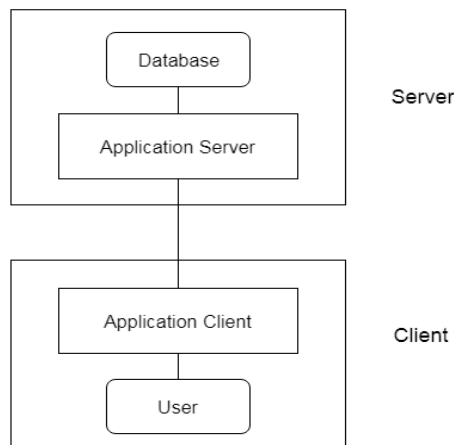


**Example:** In a banking system, a clerk or manager or cashier receives the account transactions from user/customer. Now the request of account transaction is made by customer is entered to application by clerk and request is sent to database directly. Once the request is processed in database, the result is sent back to application of clerk and transaction is completed. Here, bank employee directly communicate with database.

## 3-tier architecture:

- The 3-Tier architecture **contains another layer between the client and server**.
- In this architecture, **client can't directly communicate with the server**.
- The application on **the client-end interacts with an application server which further communicates with the database system**.
- In this architecture, **end user has no idea about the existence of the database beyond the application server**. The database also has no idea about any other user beyond the application.
- The 3-tier architecture is an extension of the 2-tier architecture, where additional Presentation or GUI Layer is added between client and server, which provides user to interact with the DBMS.
- The **3-Tier architecture is used in case of large web application**.

Following diagram shows 3-tier architecture:

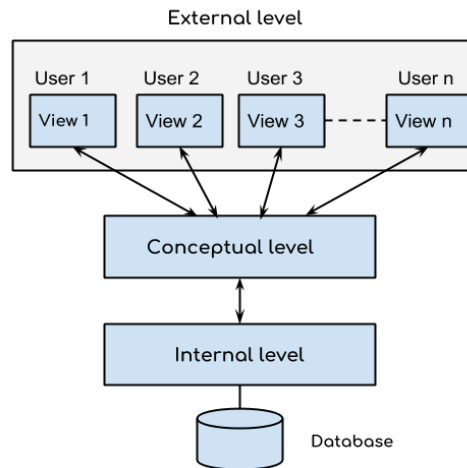


**Example:** Consider a railway ticket reservation system. A person is reserving the ticket from Delhi to Goa on particular day. Now the request of ticket reservation made by client and this request is sent to application server then that server application communicate with database and ticket issue request is store in database, the result is sent back to the client via server application to client application and ticket is reserved.

### **Three Level of DBMS architecture:**

In the previous point, we have seen the DBMS architecture – one-tier, two-tier and three-tier. In this point, we will discuss the three level DBMS architecture in detail.

Following diagram shows DBMS Three Level Architecture:



This architecture has three levels:

1. External level
2. Conceptual level
3. Internal level

#### **1. External level:**

- It is also called **view level**. The reason this level is called “view” is because **several users can only view their desired data** which is **internally fetched from database with the help of conceptual and internal level**.
- In this case user **doesn't need to know the database schema details** such as data structure, table definition etc
- Here, user is **only concerned about data which is returned back to the view which has been fetched from database** (present at the internal level).
- External level is the “top level” of the Three Level DBMS Architecture.

#### **2. Conceptual level:**

- It is also called **logical level**. The whole **design of the database such as relationship among data, schema of data etc. are described in this level**.
- **Database constraints and security** are also implemented in this level of architecture.
- This level is **maintained by DBA** (database administrator).

#### **3. Internal level:**

- This level is also known as **physical level**.
- This level describes **how the data is actually stored in the storage devices**.
- This level is also **responsible for allocating space to the data**.
- This is the **lowest level** of the architecture.

### **Schemas and Instances:**

#### ➤ **Schema:**

- The **overall design or description of a database** is called the “schema”.
- It contains a **descriptive detail of database which includes data structures, table design and relation of data from table to table based on queries etc.**
- There are three types of schemas viz. Physical schema, logical schema and view schema. Let's see these types in details.

### 1) Physical Schema:

- The **design of a database at physical level** is called “physical schema”.
- At this level, **how the data is stored in blocks of storage (like hard disc or any secondary devices) is described.**

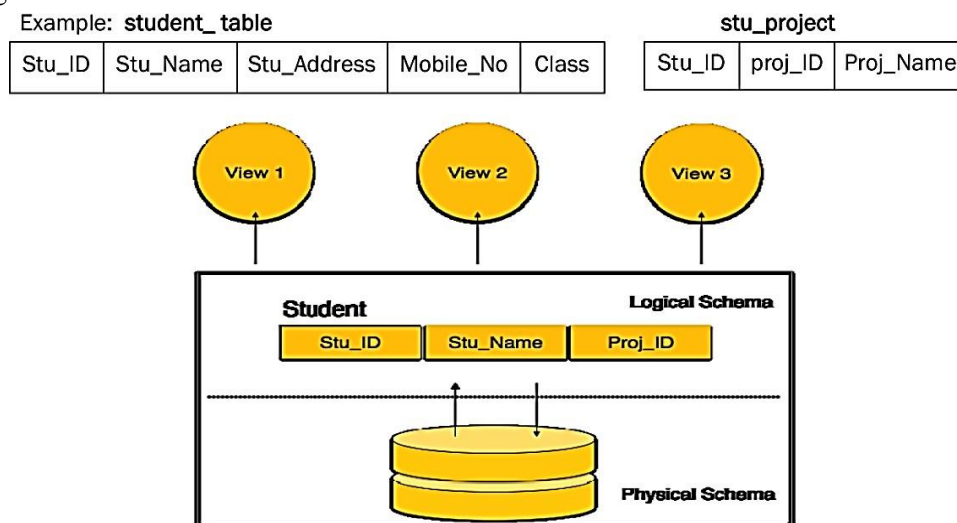
### 2) Logical Schema:

- Design of database at **logical level** is called “logical schema”.
- At this level, **which data is stored? and what is there relationship exists in all data?** is described.
- Generally, in logical schema **all table structure and the relationship between all tables** are described.
- Programmers and database administrators (DBA) work at this level.

### 3) View Schema:

- Design of database at **view level** is called “view schema”.
- This schema generally **describes end user interaction with database systems.**

Following diagram describes different schemas in database:



### Instance:

- The data stored in database at a **particular moment of time** is called “instance” of database.
- We know that, Database schema defines **the variables in tables and the value of these variables at a moment of time** is called the **instance** of that database.
- For example, let’s say we have a single table **student** in the database, today the table has 100 records, and hence today the instance of the database is 100 records. Let’s say we are going to add another 100 records by tomorrow so the instance of database will have 200 records in table

Example: Instances of Student\_table

| Stu_ID | Stu_Name | Stu_Address | Mobile_No  | Class |
|--------|----------|-------------|------------|-------|
| 3001   | Ram      | Sangola     | 9876543210 | Mca   |
| 4041   | Laxman   | Pune        | 7896543218 | Msc   |

#### • Difference between schema and instance:

| Schema                                                   | Instance                                                                                |
|----------------------------------------------------------|-----------------------------------------------------------------------------------------|
| It is the overall description or design of the database. | It is the collection of information stored in a database at a particular moment of time |
| Schema is same for whole database.                       | Data in instances can be changed using addition, deletion and updation.                 |
| Does not change Frequently.                              | Changes Frequently.                                                                     |

|                                                                                              |                                                           |
|----------------------------------------------------------------------------------------------|-----------------------------------------------------------|
| Defines the basic structure of the database i.e how the data will be stored in the database. | It is the set of Information stored at a particular time. |
|----------------------------------------------------------------------------------------------|-----------------------------------------------------------|

## Database languages:

Database languages are **set of commands used to create database structure, insert and retrieve data, control user access and manage transactions** in a database.

### Types of database languages:

- 1) Data Definition Language (DDL)
- 2) Data Manipulation Language (DML)
- 3) Data Control Language (DCL)
- 4) Transaction Control Language (TCL)

Let's see these languages in details-

#### 1) Data Definition Language (DDL):

- The DDL are used to **define and modify the structure of database objects** (Table, Index, view, sequence etc.)
- The DDL statements or commands are –
  - CREATE: This command is **used to create new database object** (table, index, view etc.)
  - ALTER: This command is **used to alter or modify existing database object**.
  - DROP: This command is **used to drop or remove the database object**.
  - RENAME: This command is **used to set a new name (i.e. change the existing name) for database object**.
  - TRUNCATE: This command is **used to removes all the records or tuples from a table. But this command will not destroy the table's structure.**

#### 2) Data Manipulation Language (DML):

- DML language is used to manipulate i.e. manage and change the data stored in database like inserting, updating and deleting the data stored in table, also retrieving record from a table, etc.
- DML allows user to insert, delete, update and select (retrieve) data from database.
- The DML statements or commands are –
  - SELECT: This command is **used to retrieve data from the database** tables.
  - INSERT: This command is **used to insert data into database** table.
  - UPDATE: This command is **used to update or change the data** of database table.
  - DELETE: This command is **used to delete specified rows (one or more) from the table that satisfied specific constraint or condition**.

#### 3) Data Control Language (DCL):

- DCL language is used to give the permission (GRANT) for database user to use the database or database objects.
- It can also **take back the permission (REVOKE) given to database user**.
- It allows central administration of user privileges and access to database.
- The DCL statements or commands are –
  - GRANT: This command is used to give privilege to access the database.
  - REVOKE: This command is used to take back the privilege to access the database.

#### 4) Transaction Control Language (TCL):

- TCL language used to **manage transactions (commands of DML)** in the Database.
- The TCL statements or commands are –
  - COMMIT: This command is used to **save the database work (DML result) permanently into database**.
  - ROLLBACK: This command is **used to restores database work since last commit**.
  - SAVEPOINT: This command is **used to set a point in transaction to roll back later**.

## Data Independence:

- Data Independence is defined as; it is a property of DBMS that **helps you to change the database schema at one level of a database system without affecting (requiring to change) the schema at the next higher level.** (since, there are three level of schemas viz Physical, logical and view )
- If a database system is **not multi-layered** (multi-level), then it becomes difficult to make any changes in the database system. But with the help of data independence, making any changes in the database becomes easy.
- Data independence helps you to keep data separated from all programs of system.

## Types of Data Independence:

There are two types of Data Independence namely-

- 1) Physical Data independence
- 2) Logical Data independence

Let's see these types in details-

### 1) Physical Data Independence

- Ability to change **physical storage/structure** without affecting higher-level schemas.
- Actual data is stored in bits on disk; physical changes should not impact **logical or view levels**.
- Achieved through the **internal level** of DB and transformation between **conceptual ↔ internal level**.

---

### 2) Logical Data Independence

- Ability to change **logical structure (conceptual schema)** without affecting **views or application programs**.
- Modifications in logical level do not affect **external views or APIs**.
- Changes in table format should not impact **data storage** on disk.

## Difference between Physical Data Independence and Logical Data Independence:

| Logical Data Independence                                                                                         | Physical Data Independence                                                                                         |
|-------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------|
| 1) Logical Data Independence is mainly concerned with the structure or changing the data definition.              | 1) Mainly concerned with the storage of the data.                                                                  |
| 2) It is difficult as the retrieving of data is mainly dependent on the logical structure of data.                | 2) It is easy to retrieve the data.                                                                                |
| 3) Compared to Physical independence it is difficult to achieve logical data independence.                        | 3) Compared to Logical Independence it is easy to achieve physical data independence.                              |
| 4) You need to make changes in the Application program if new fields are added or deleted from the database.      | 4) A change in the physical level usually does not need change at the Application program level.                   |
| 5) Modification at the logical levels is significant whenever the logical structures of the database are changed. | 5) Modifications made at the internal levels may or may not be needed to improve the performance of the structure. |
| 6) It concerned with conceptual schema                                                                            | 6) It concerned with internal schema                                                                               |
| Example: Add/Modify/Delete a new attribute                                                                        | Example: change in compression techniques, hashing algorithms, storage devices, etc                                |

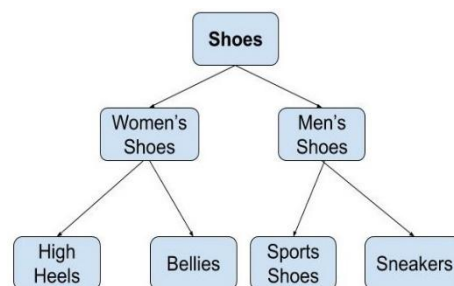
## Data Model:

- Data Models are used to show how data is stored, connected, accessed and updated in the database management system.
- Data Model gives us an idea that how the final system will look like after its complete implementation.
- It defines the data elements and the relationships between the data elements etc.
- In data model, we use a set of symbols and text to represent the information so that members of the organization can communicate and understand it.
- Though there are many data models being used, but now a day the Relational model is the most widely used model. Apart from the Relational model, there are many other types of data models available, but we will study several data model listed below. Some of the Data Models in DBMS are:
  - 1) Hierarchical Model
  - 2) Network Model
  - 3) Entity-Relationship Model
  - 4) Relational Model
  - 5) Hybrid Model

Let's see all these models in details-

### 1) Hierarchical Model:

- Hierarchical Model is the first DBMS model.
- This model organizes the data in the hierarchical or tree structure manner.
- The hierarchy starts from the root and then it expands in the form of a child nodes to the parent node.
- This model easily represents some of the real-world relationships like food recipes, sitemap of a website etc.  
Example: We can represent the relationship between the shoes present on a shopping website in the following way:



*Hierarchical Model*

### Features / Advantages of Hierarchical Model

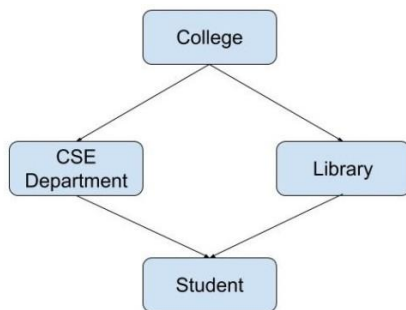
- **One-to-many relationship:** Data organized in tree structure; only one path from parent to child.
- **Parent-Child relationship:** One parent, many children; multiple parents not allowed.
- **Automatic deletion:** Deleting parent deletes its child nodes.
- **Pointers:** Link parent and child nodes; used for navigation.
- **Simple & fast traversal** of tree.
- **Data integrity maintained:** Changes in parent auto-reflect in child.

### Disadvantages of Hierarchical Model

- Cannot represent **complex relationships**.
- Does not support **multiple parents** for a child.
- **Deletion issue:** Deleting parent deletes child nodes.

### 2) Network Model:

- This model is an extension of the hierarchical model.
- It was the most popular model before the relational model.
- This model is the same as the hierarchical model, the only difference is that a record can have more than one parent.
- It replaces the hierarchical tree with a graph.
- Example: In the example below we can see that node student has two parents i.e. CSE Department and Library. This was earlier not possible in the hierarchical model.



Network Model

### Features / Advantages of Network Model

- Supports **one-to-one & many-to-many relationships**.
- **Fast & simple data access** (multiple paths to same record).
- Uses **circular linked list** for navigation.
- Faster than hierarchical model (multiple access paths).
- **Data integrity**: Changes in parent reflect in child.

### Disadvantages of Network Model

- **Complex system** due to many relationships.
- **Updation, deletion, insertion** operations are complex.

## 3) Entity-Relationship Model (E-R Model):

- Entity-Relationship Model or simply ER Model is a high-level data model.
- In this model, we represent the **real-world problem in the pictorial form to make it easy for the stakeholders or investors to understand**.
- It is also very **easy for the developers to understand the system by just looking at the ER diagram**.
- We use the **ER diagram as a visual tool to represent an ER Model**.
- **ER diagram** has the following three components:
  - 1) Entity
  - 2) Attribute
  - 3) Relationship

Let's see these in details-

**1) Entities:** Entity is a **real-world thing. It can be a person, place, or even any concept**.

**Example:** Teachers, Students, Course, Building, Department etc. are some of the entities of a School Management System.

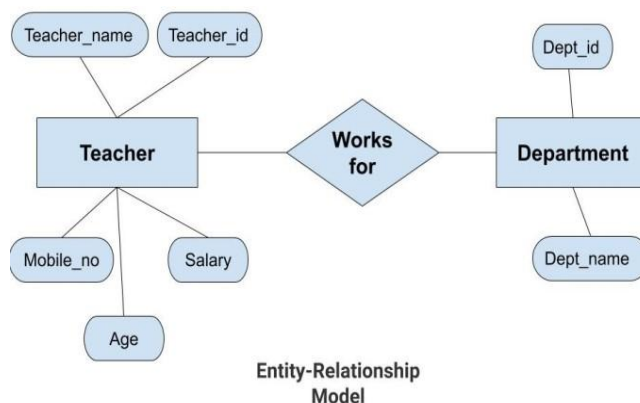
**2) Attributes:** A real-world property or characteristics of an entity is called 'attribute'.

**Example:** The entity teacher has the attributes or properties like teacher id, name, salary, age, etc.

**3) Relationship:** Relationship tells how two entities are related.

**Example:** Teacher works for a department.

Following diagram shows E-R Model-



Entity-Relationship Model

In the above diagram, the entities are Teacher and Department. The attributes of Teacher entity are Teacher\_Name, Teacher\_id, Age, Salary, Mobile\_Number. The attributes of entity Department entity are Dept\_id, Dept\_name. The two entities are connected using the relationship (works for). Here, each teacher **works for** a department.

## Features / Advantages of ER Model

- **Graphical & simple:** Easy to understand and use.
- **ER diagram:** Visual tool for representation.
- **Database design aid:** Helps in building database structure.
- **Effective communication** between developers & stakeholders.
- **Easy conversion** to relational, network, or hierarchical models.

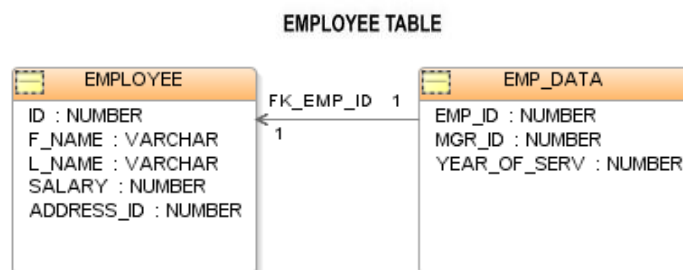
## Disadvantages of ER Model

- **No standard notation** (developers may use different symbols).
- **Hidden information:** Some details may be lost in high-level view.

## 4) Relational Model:

- Relational database is probably the most popular and widely **used data model for storing and retrieving data since 1970's**.
- Developers find it simple and flexible for use, also because of its **data abstraction feature** which hide its internal data storage complexity from users.
- In this model, the data is maintained in the form of a two-dimensional array i.e. table.
- All the information is stored in the form of row and columns.
- The **basic structure of a relational model** is **tables**. So, the **tables** are also called **relations** in the relational model.
- Example: In this example, we have an Employee table.

| Emp_id    | Emp_name | Job_name | Salary | Mobile_no  | Dep_id | Project_id |
|-----------|----------|----------|--------|------------|--------|------------|
| AfterA001 | John     | Engineer | 100000 | 9111037890 | 2      | 99         |
| AfterA002 | Adam     | Analyst  | 50000  | 9587569214 | 3      | 100        |
| AfterA003 | Kande    | Manager  | 890000 | 7895212355 | 2      | 65         |



**Relation Between EMPLOYEE & EMP\_DATA  
Tables**

## Features / Advantages of Relational Model

- **Tuples (rows):** Represent instances of objects.
- **Attributes (fields):** Properties/columns defining relations.
- **Simple:** Easier than network & hierarchical models.
- **Scalable:** Rows & columns can be easily added.
- **Structural independence:** DB structure changes don't affect data access.

## Disadvantages of Relational Model

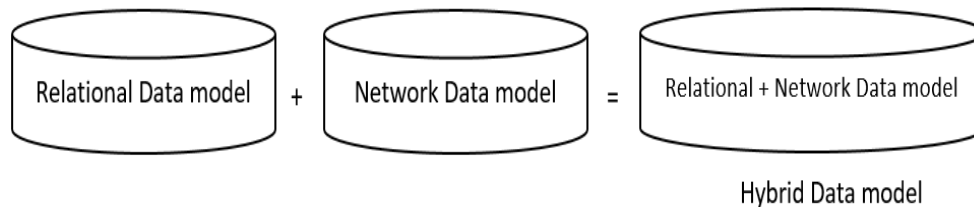
- Needs **powerful hardware** for handling complexity.
- Poor design may cause **slow performance** as DB grows

But all these disadvantages are minor as compared to the advantages of the relational model. These problems can be avoided with the help of proper implementation and organisation.

## 5) Hybrid Model:

- As the name implies, Hybrid data model is **combination of two different types of data models**.
- It may be combination of relational with network model or relational with hierarchical model etc.
- In hybrid data model of relational with network model can have advantages of both models. That is this hybrid model can easy to maintain relations (Since, it is relational) & can access the data fast (Since, it is network).
- Implementaion of such hybrid model is complex task.
- Also, to manipulate data of such hybrid data model requires to develop separate query language that supports both relational data model & network data model.
- To develop such separate query language, expert developers/programmer needed and that makes it costly.

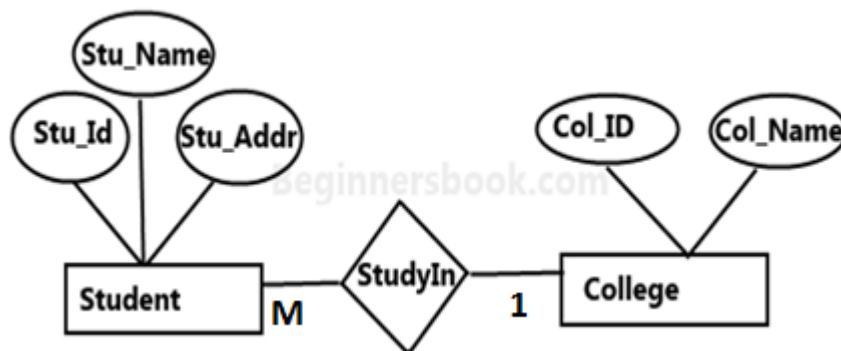
• **Following diagram shows Hybrid data model-**



## E-R Model (Entity Relationship Model):

- An Entity–relationship model (ER model) describes the structure of a database with the help of a diagram, which is known as Entity Relationship Diagram (ER Diagram).
- An ER model is a design or blueprint of a database that can later be implemented as a database.
- ER model stands for an Entity-Relationship model. It is a high-level data model.
- Basically this model is used to define the data elements and relationship for a specified system.
- It develops a conceptual design for the database. It also develops a very simple and easy to design view of data.

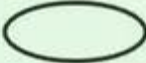
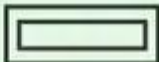
**For example,** In the following diagram we have two entities Student and College and their relationship. Student entity has attributes such as Stu\_Id, Stu\_Name & Stu\_Addr and College entity has attributes such as Col\_ID & Col\_Name. The relationship between Student and College is many to one as a college can have many students however a student cannot study in multiple colleges at the same time.



## Geometrical shapes used in the construction of ER-Model / ER-diagram:

Here are the geometric shapes and their meaning used in an E-R Diagram.

1. Rectangle: Represents Entity sets.
2. Ellipses or Oval: Attributes
3. Diamonds: Relationship Set
4. Lines: They link attributes to Entity Sets and Entity sets to Relationship Set
5. Double Ellipses: Multivalued Attributes
6. Dashed Ellipses: Derived Attributes
7. Double Rectangles: Weak Entity Sets
8. Double Lines: Total participation of an entity in a relationship set

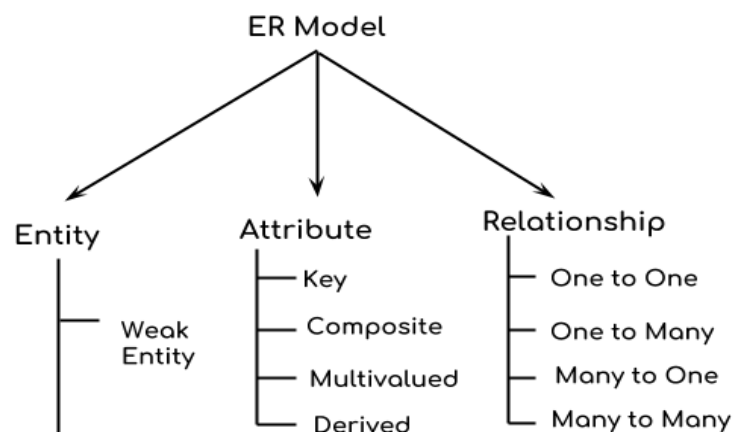
| Figures          | Symbols                                                                           | Represents                                                           |
|------------------|-----------------------------------------------------------------------------------|----------------------------------------------------------------------|
| Rectangle        |  | Entities in ER Model                                                 |
| Ellipse          |  | Attributes in ER Model                                               |
| Diamond          |  | Relationships among Entities                                         |
| Line             |  | Attributes to Entities and Entity Sets with Other Relationship Types |
| Double Ellipse   |  | Multi-Valued Attributes                                              |
| Double Rectangle |  | Weak Entity                                                          |

### Components of ER- diagram:

ER-diagram has three basic components-

- 1) Entity
- 2) Attribute
- 3) Relationship

Following diagram shows components of ER- diagram (ER-Model):



Let's see all these components in details-

#### 1) Entity:

- Entity is a real-world thing. It can be represent any person, place, or even any concept.
- An entity is represented as rectangle in an ER diagram.

Example: Teachers, Students, Course, Building, Department etc. are some of the entities of a School Management System.

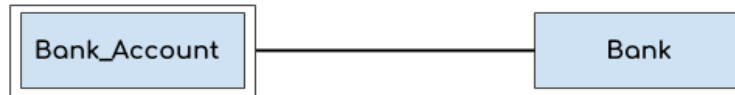
Following ER-diagram has two entities namely student and college. And these entities are in many to one relationship as many students are study in one college.



#### Weak Entity:

- An entity that cannot be uniquely identified by its own attributes and it depend on (relies) the relationship with other entity is called 'weak entity'.
- The weak entity is represented by a double rectangle.

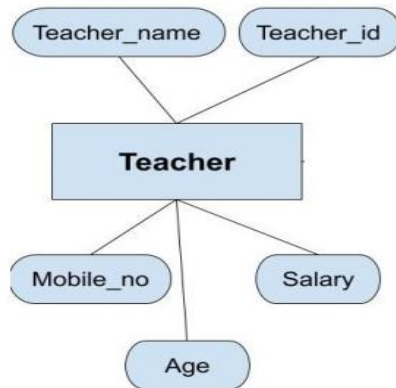
- For example – a bank account cannot be uniquely identified without knowing the bank to which the account belongs, so bank account is a weak entity.



## 2) Attributes:

- A property or characteristics of an entity is called 'attribute'.
- An attribute is represented as Oval or ellipse in an ER diagram

Example: The entity teacher has the attributes or properties like Teacher\_id, Teacher\_name, salary, age, mobile\_no.



## Types of Attributes:

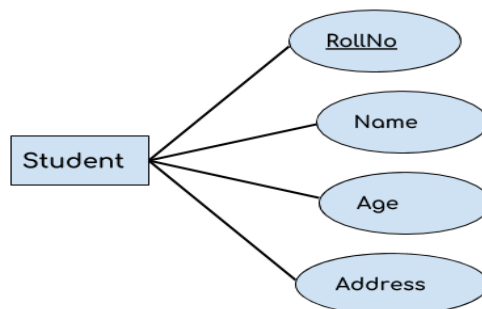
There are four types of attributes:

1. Key attribute
2. Composite attribute
3. Multivalued attribute
4. Derived attribute

Let's see all these in details-

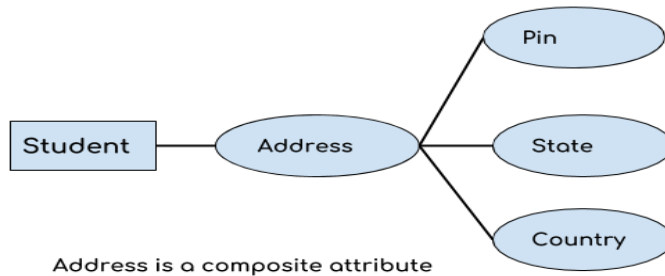
### 1) Key attribute:

- A key attribute can uniquely identify an entity from an entity set.
- Key attribute is represented by oval same as other attributes however the **text of key attribute is underlined**.
- For example, student roll number can uniquely identify a student from a set of students.



### 2) Composite attribute:

- An attribute that is a combination of other attributes is known as 'composite attribute'.
- For example, In student entity, the student address is a composite attribute as an address is composed of other attributes such as pin code, state, country.



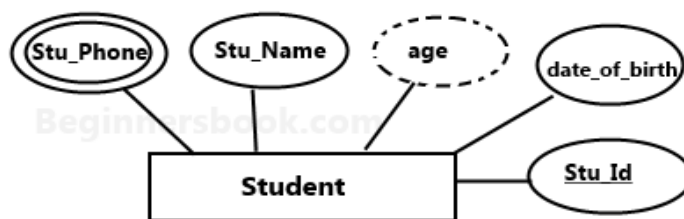
### 3) Multivalued attribute:

- An attribute that can hold multiple values is known as 'multivalued attribute'.
- It is represented with double ovals in an ER Diagram.
- For example – A person can have more than one phone numbers so the phone number attribute is multivalued.

### 4) Derived attribute:

- A derived attribute is one whose value is dynamic and derived from another attribute.
- It is represented by dashed oval in an ER Diagram.
- For example – Person age is a derived attribute as it changes over time and can be derived from another attribute (Date of birth).

E-R diagram with multivalued and derived attributes:



### 3) Relationship:

- Relationship tells how two entities are related or associated with each other.
- A relationship is represented by diamond shape in ER diagram.

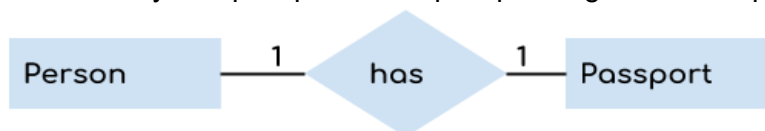
There are four types of relationships:

1. One to One (1:1)
2. One to Many (1: M)
3. Many to One (M: 1)
4. Many to Many (M: M)

Let's see this relationship in details-

#### 1. One to One Relationship:

- When a single instance of an entity is associated with a single instance of another entity then it is called 'one to one relationship'.
- For example, a person has only one passport and a passport is given to one person.



#### 2. One to Many Relationship:

- When a single instance of an entity is associated with more than one instances of another entity then it is called 'one to many relationship'.
- For example – a customer can place many orders but an order cannot be placed by many customers.



### 3. Many to One Relationship:

- When more than one instances of an entity is associated with a single instance of another entity then it is called 'many to one relationship'.
- For example – many students can study in a single college but a student cannot study in many colleges at the same time.



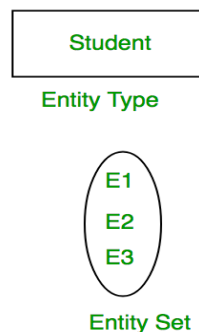
### 4. Many to Many Relationship:

- When more than one instances of an entity is associated with more than one instances of another entity then it is called 'many to many relationship'.
- For example, many student can be assigned to many projects.



### Entity Type and Entity set:

- We know that an Entity is real thing that may represent with a physical existence like any person, car, house, or employee.
  - Or it may be an object with a conceptual existence – a company, a job, or a university course.
  - Or we also say that an Entity is an object of Entity Type and set of all entities is called as 'Entity set'.
- Example- In following ER-diagram, E1, E2, E3 are entities having Entity Type Student and set of all such students entities is called Entity Set.



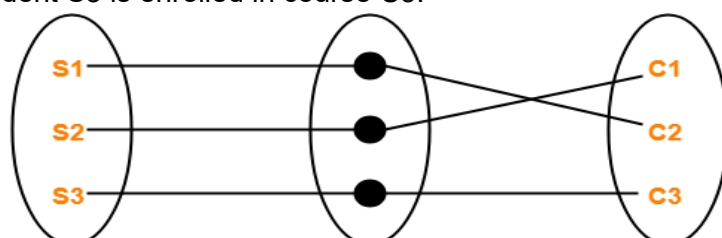
### Relationship Type and Relationship Set:

- A relationship type represents the association between entity types.
- In ER diagram, relationship type is represented by a diamond and connecting the entities with lines.
- In following diagram 'Enrolled in' is a relationship type that exists between entity type Student and Course.



### Relationship set:

- A set of relationships of same type is known as 'relationship set'.
- The following relationship set shows student S1 is enrolled in course C2, student S2 is enrolled in course C1 and student S3 is enrolled in course C3.

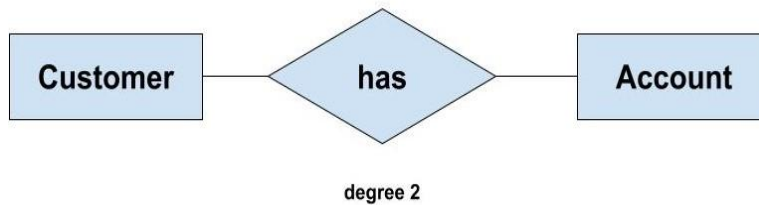


### Degree of Relationship set:

- The number of different entity sets participating in a relationship set is called as 'degree of a relationship set'.

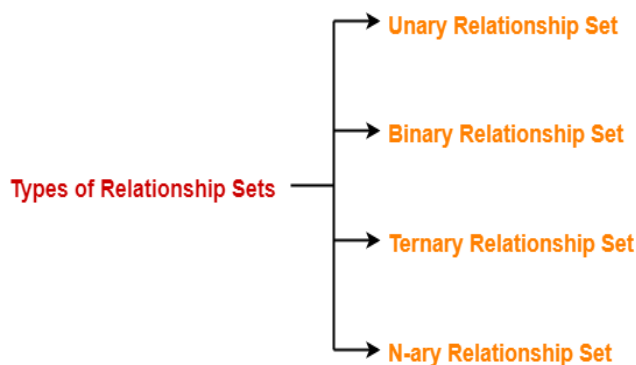
**Degree of a relationship set = Number of entity sets participating in a relationship set**

- For example, If we have two entity type 'Customer' and 'Account' and they are linked using the primary key and foreign key. We can say that the degree of relationship is 2 because here two entities are taking part in the relationship.



### Types of Relationship set:

On the basis of degree of a relationship set, a relationship set can be classified into the following types-

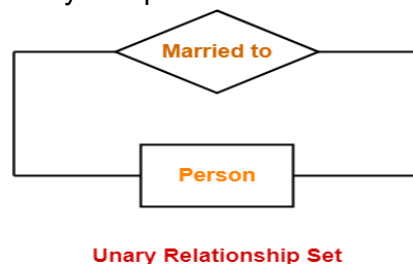


- 1) Unary relationship set
- 2) Binary relationship set
- 3) Ternary relationship set
- 4) N-ary relationship set

Let's see all these types in details-

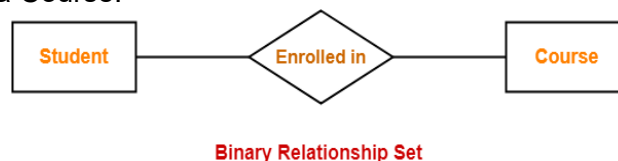
#### 1) Unary relationship set:

- Unary relationship set is a relationship set where only one entity set participates in a relationship set.
- Ex- One person is married to only one person.



#### 2) Binary relationship set:

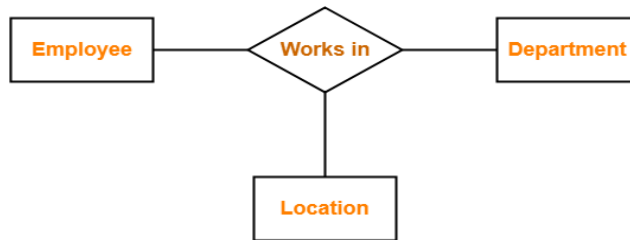
- Binary relationship set is a relationship set where two entity sets participate in a relationship set.
- Ex- Student is enrolled in a Course.



#### 3) Ternary relationship set:

- Ternary relationship set is a relationship set where three entity sets participate in a relationship set.

Ex-

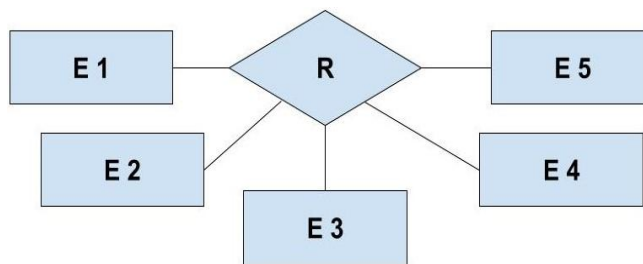


**Ternary Relationship Set**

#### 4) N-ary relationship set:

- N-ary relationship set is a relationship set where 'n' number of entity sets participate in a relationship set. Here  $n > 3$ .

Ex- In following diagram, there are 5 number of entity sets participate in a relationship set that's why it is n-ary relationship set.



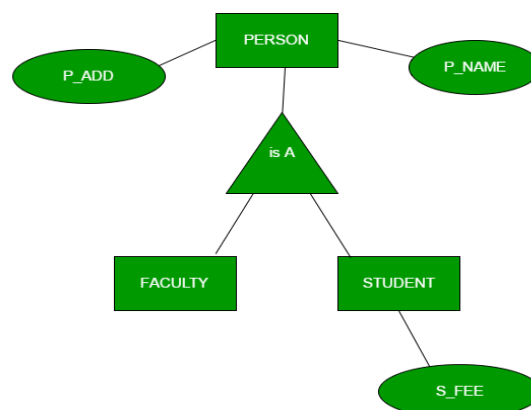
**N-ary Relationship**

### Generalization, Specialization and Aggregation in ER Model:

- Generalization, Specialization and Aggregation in ER model are used for data abstraction in which abstraction mechanism is used to hide details of a set of objects.

#### Generalization –

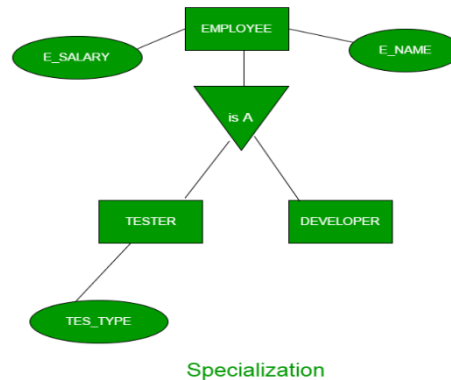
- Generalization is the process of extracting common properties from a set of entities and create a generalized entity from it.
- It is a bottom-up approach in which two or more entities can be generalized to a higher level entity if they have some attributes in common.
- For Example, STUDENT and FACULTY can be generalized to a higher level entity called PERSON as shown in Figure. In this case, common attributes like P\_NAME, P\_ADD become part of higher entity (PERSON) and specialized attributes like S\_FEE become part of specialized entity (STUDENT).



#### Specialization –

- In specialization, an entity is divided into sub-entities based on their characteristics.
- It is a top-down approach where higher level entity is specialized into two or more lower level entities.

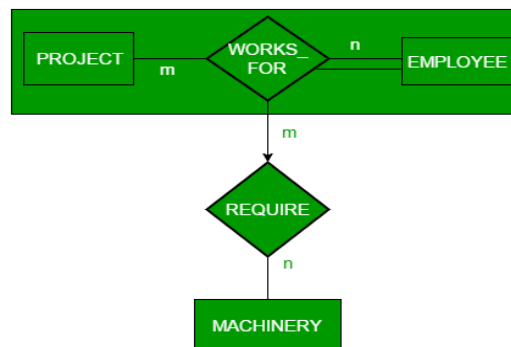
- For Example, EMPLOYEE entity in an Employee management system can be specialized into DEVELOPER, TESTER etc. as shown in Figure. In this case, common attributes like E\_NAME, E\_SAL etc. become part of higher entity (EMPLOYEE) and specialized attributes like TES\_TYPE become part of specialized entity (TESTER).



Specialization

## Aggregation:

- Aggregation is an abstraction through which we can represent relationships as higher level entity sets.
- An ER diagram is not capable of representing relationship between an entity and a relationship which may be required in some scenarios or in some situation. In those cases, a relationship with its corresponding entities is aggregated into a higher-level entity.
- For Example, Employee working for a project may require some machinery. So, REQUIRE relationship is needed between relationship WORKS\_FOR and entity MACHINERY. Using aggregation, WORKS\_FOR relationship with its entities EMPLOYEE and PROJECT is aggregated into single entity and relationship REQUIRE is created between aggregated entity and MACHINERY.



Aggregation

## What is an RDBMS?

**RDBMS** stands for **Relational Database Management System**.

It is a type of database management system that stores data in a **structured format**, using **rows and columns** (i.e., tables). RDBMSs make it easy to **organize, access, and manage data** efficiently.

### Key Concepts of RDBMS:

- Tables:** Data is stored in **tables**, which consist of **rows** (records) and **columns** (fields). Each table represents one entity (e.g., Customers, Orders).
- Rows:** Each **row** is a single record or entry (e.g., one customer's details).
- Columns:** Each **column** represents a data field (e.g., Name, Email, Age).
- Primary Key:** A column (or combination of columns) that **uniquely identifies** each row in a table.
- Foreign Key:** A column that creates a **relationship between two tables** by referencing the primary key of another table.
- SQL (Structured Query Language):** The standard language used to **interact with RDBMSs** – for querying, updating, and managing data.

### Examples of Popular RDBMS Software:

- MySQL**

- PostgreSQL
- Oracle Database
- Microsoft SQL Server
- SQLite

#### Features or Advantages of RDBMS:

- Data integrity and accuracy through constraints.
- Easy to retrieve and manipulate data using SQL.
- Supports data relationships via keys.
- Allows multiple users to access data concurrently.
- Secure, with access controls and permissions.

#### Uses of RDBMS:

RDBMSs are widely used in various industries for:

- Banking and financial systems
- E-commerce platforms
- Enterprise applications (ERP, CRM)
- Education and healthcare systems

#### Difference between DBMS and RDBMS:

| Feature / Criteria         | DBMS (Database Management System)                           | RDBMS (Relational Database Management System)                            |
|----------------------------|-------------------------------------------------------------|--------------------------------------------------------------------------|
| <b>Data Storage</b>        | Stores data in <b>files</b> or <b>non-tabular</b> formats   | Stores data in <b>tables (rows and columns)</b>                          |
| <b>Data Relationships</b>  | Does <b>not support</b> relationships between data elements | <b>Supports relationships</b> using <b>keys</b> (primary & foreign keys) |
| <b>Data Integrity</b>      | Low level of data integrity                                 | High level of data integrity using constraints                           |
| <b>Normalization</b>       | <b>Not supported</b>                                        | <b>Supported</b> to eliminate redundancy                                 |
| <b>Query Language</b>      | May not use SQL                                             | Uses <b>Structured Query Language (SQL)</b>                              |
| <b>Data Redundancy</b>     | High                                                        | Low, due to normalization and relationships                              |
| <b>Security Features</b>   | Basic or limited                                            | Advanced security features (e.g., user roles, permissions)               |
| <b>Concurrency Control</b> | Minimal                                                     | Supports <b>multi-user environment</b> with <b>concurrency control</b>   |
| <b>Examples</b>            | File systems, XML databases                                 | MySQL, Oracle, PostgreSQL, SQL Server                                    |
| <b>Complexity and Cost</b> | Simpler and usually cheaper                                 | More complex and often costlier (but more powerful)                      |

#### 12 Codd's rules: (E.F. Codd's 12 Rules)

The E.F. Codd's 12 Rules are a set of guidelines proposed by Dr. Edgar F. Codd, the inventor of the relational database model, to define **what is required from a Database Management System (DBMS) for it to be considered a Relational Database Management System (RDBMS)**.

Following table shows Codd's 12 rules-

| Rule No.      | Rule Name                     | Description                                                                                                 |
|---------------|-------------------------------|-------------------------------------------------------------------------------------------------------------|
| <b>Rule 0</b> | <b>Foundation Rule</b>        | A system must use relational capabilities to manage a database and must support all the other rules.        |
| <b>Rule 1</b> | <b>Information Rule</b>       | All information in a database is represented <b>only</b> as values in <b>tables</b> (columns and rows).     |
| <b>Rule 2</b> | <b>Guaranteed Access Rule</b> | Each data item (value) should be accessible by specifying <b>table name, primary key, and column name</b> . |

|                |                                              |                                                                                                                                                      |
|----------------|----------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Rule 3</b>  | <b>Systematic Treatment of Nulls</b>         | The DBMS must support a <b>consistent representation and handling of NULL values</b> , regardless of data type.                                      |
| <b>Rule 4</b>  | <b>Dynamic Online Catalog</b>                | The database catalog (metadata) must be stored in the same way as regular data, i.e., in <b>tables</b> , and can be queried using SQL.               |
| <b>Rule 5</b>  | <b>Comprehensive Data Sub-language Rule</b>  | The system must support at least one language (like <b>SQL</b> ) that can define data, manipulate data, define constraints, and handle transactions. |
| <b>Rule 6</b>  | <b>View Updating Rule</b>                    | All views (virtual tables) that can be updated logically must also be updatable by the system.                                                       |
| <b>Rule 7</b>  | <b>High-Level Insert, Update, and Delete</b> | The system must support set-level (not just row-level) operations for insert, update, and delete.                                                    |
| <b>Rule 8</b>  | <b>Physical Data Independence</b>            | Changes in physical storage (e.g., file structure) should not affect how data is accessed or managed by users.                                       |
| <b>Rule 9</b>  | <b>Logical Data Independence</b>             | Changes in the logical structure (e.g., adding a column) should not require changes in applications.                                                 |
| <b>Rule 10</b> | <b>Integrity Independence</b>                | Integrity constraints (like primary keys, foreign keys, etc.) should be stored in the catalog and not in application code.                           |
| <b>Rule 11</b> | <b>Distribution Independence</b>             | The user should not be aware of whether the database is distributed across multiple locations.                                                       |
| <b>Rule 12</b> | <b>Non-subversion Rule</b>                   | If the system provides a low-level access method, it must not bypass the integrity rules and constraints.                                            |

## Normalization:

- **Normalization** is the process of **organizing data in a database** to reduce **data redundancy** and improve **data integrity**.
- It involves **dividing large tables into smaller**, related tables and defining relationships between them.
- The goal is to:
  - Eliminate duplicate data
  - Ensure data dependencies make sense
  - Improve consistency and efficiency of data operations

### Normalization Forms in Tabular Format:

| <b>Normal Form</b> | <b>Full Name</b>          | <b>Condition / Rule</b>                                                                                                       | <b>Purpose</b>                                        |
|--------------------|---------------------------|-------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------|
| <b>1NF</b>         | First Normal Form         | - All values in a column must be <b>atomic</b> (indivisible) - No repeating groups or arrays                                  | Eliminates repeating groups and ensures atomic values |
| <b>2NF</b>         | Second Normal Form        | - Must be in 1NF - All <b>non-key attributes</b> must be <b>fully functionally dependent</b> on the <b>entire primary key</b> | Removes partial dependency (in composite keys)        |
| <b>3NF</b>         | Third Normal Form         | - Must be in 2NF - <b>No transitive dependencies</b> (non-key attributes must not depend on other non-key attributes)         | Removes transitive dependency                         |
| <b>BCNF</b>        | Boyce-Codd Normal Form    | - Must be in 3NF - For every <b>functional dependency</b> , the <b>left side must be a super key</b>                          | Handles certain edge cases not covered by 3NF         |
| <b>4NF</b>         | Fourth Normal Form        | - Must be in BCNF - No <b>multi-valued dependencies</b>                                                                       | Eliminates multi-valued dependencies                  |
| <b>5NF</b>         | Fifth Normal Form (PJ/NF) | - Must be in 4NF - No <b>join dependency</b> exists that is not implied by candidate keys                                     | Ensures lossless join of decomposed tables            |

**Practical example that demonstrates step-by-step normalization from Unnormalized Form (UNF) to Fifth Normal Form (5NF).**

---

### Scenario: Student-Course Enrollment Data

#### Unnormalized Table (UNF)

| StudentID | StudentName | Courses          | Instructors           |
|-----------|-------------|------------------|-----------------------|
| 1         | Alice       | Math,<br>Science | Mr. John,<br>Ms. Kate |
| 2         | Bob         | Math             | Mr. John              |

Here, Problems are:

- Courses and Instructors contain **multiple values (non-atomic)**.
  - This is **not in 1NF**.
- 

#### Step 1: First Normal Form (1NF)

 Make all columns atomic (no multivalued attributes):

| StudentID | StudentName | Course  | Instructor |
|-----------|-------------|---------|------------|
| 1         | Alice       | Math    | Mr. John   |
| 1         | Alice       | Science | Ms. Kate   |
| 2         | Bob         | Math    | Mr. John   |

 Now all columns contain **atomic values**, and the table is now in **1NF**.

---

#### Step 2: Second Normal Form (2NF)

 Requirements:

- Must be in **1NF**
  - **No partial dependency** on part of a **composite primary key**
- Assuming composite key: (StudentID, Course)
- StudentName depends only on StudentID → partial dependency

 Split into two tables:

 Students Table:

| StudentID | StudentName |
|-----------|-------------|
| 1         | Alice       |
| 2         | Bob         |

 Enrollments Table

| StudentID | Course  | Instructor |
|-----------|---------|------------|
| 1         | Math    | Mr. John   |
| 1         | Science | Ms. Kate   |
| 2         | Math    | Mr. John   |

 Now, all **non-key attributes depend on the full key** that implies tables now in **2NF**

---

#### Step 3: Third Normal Form (3NF)

 Requirements:

- Must be in **2NF**
- **No transitive dependency** (i.e., non-key → non-key)

In Enrollments, Instructor depends on Course, not on (StudentID, Course)

 Further split the tables:

### Enrollments Table

| StudentID | Course  |
|-----------|---------|
| 1         | Math    |
| 1         | Science |
| 2         | Math    |

### Course-Instructor Table

| Course  | Instructor |
|---------|------------|
| Math    | Mr. John   |
| Science | Ms. Kate   |

✓ All **non-key attributes** now depend **only on the primary key**, and no transitive dependencies that implies tables in **3NF**

---

### ✓ Step 4: Boyce-Codd Normal Form (BCNF)

👉 Additional to 3NF:

- For every functional dependency  $X \rightarrow Y$ , **X must be a super key**
- Our tables already follow this:
- In Enrollments, (StudentID, Course) is the key.
  - In Course-Instructor, Course is the key for Instructor.

✓ No violations that implies tables are also in **BCNF**

---

### ✓ Step 5: Fourth Normal Form (4NF)

👉 No **multi-valued dependencies** (MVDs)

Let's say a student can be enrolled in multiple **courses** and be involved in multiple **projects** independently:

| StudentID | Course  | Project  |
|-----------|---------|----------|
| 1         | Math    | AI       |
| 1         | Science | AI       |
| 1         | Math    | Robotics |
| 1         | Science | Robotics |

Here, two **independent MVDs** exist:

- $StudentID \twoheadrightarrow Course$
- $StudentID \twoheadrightarrow Project$

 Split into:

**Enrollments:**

| StudentID | Course  |
|-----------|---------|
| 1         | Math    |
| 1         | Science |

**Student\_Projects:**

| StudentID | Project  |
|-----------|----------|
| 1         | AI       |
| 1         | Robotics |

✓ Now tables are in **4NF**

---

### ✓ Step 6: Fifth Normal Form (5NF / PJNF)

👉 Eliminate **join dependencies** without loss of data.

Imagine a case where a **student can be taught by multiple instructors for multiple courses**:

| StudentID | Course  | Instructor |
|-----------|---------|------------|
| 1         | Math    | Mr. John   |
| 1         | Math    | Ms. Kate   |
| 1         | Science | Mr. John   |
| 1         | Science | Ms. Kate   |

This data can be **factored** into 3 separate projections:

1. Student-Course
2. Course-Instructor
3. Student-Instructor

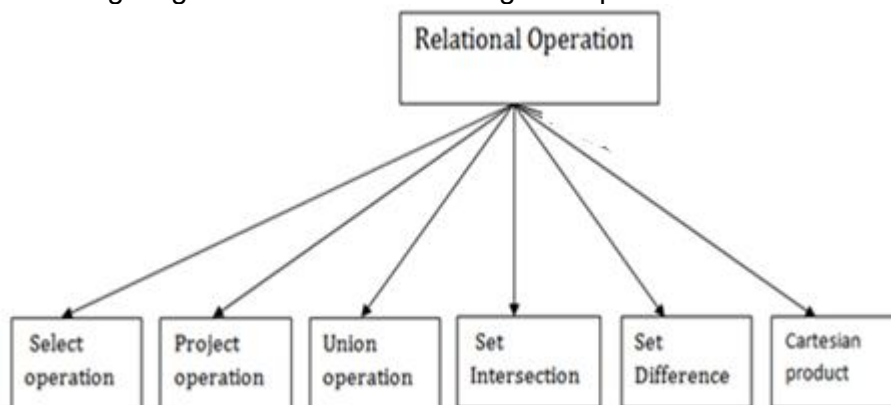
✓ 5NF ensures that such decomposed tables can be **joined without data loss**, and no extra data is introduced.

## Relational Algebra:

- Relational algebra is a procedural query language, which takes instances of relations as input and yields instances of relations as output.
- Relational algebra uses operators to perform queries. An operator can be either unary or binary. They accept relations as their input and yield relations as their output. Relational algebra is performed recursively on a relation and produce results which are also considered as relations.
- It gives a step by step process to obtain the result of the query.

## Relational Algebra Operations:

Following diagram shows Relational algebra operations-



Let's see all these operations in details-

### 1) Select Operation:

- The select operation selects tuples or record that satisfy a given predicate or condition.
- It is denoted by sigma ( $\sigma$ ).

Notation:  $\sigma p(r)$

Where:

$\sigma$  is used for selection prediction

$r$  is used for relation

$p$  is used as a propositional logic formula which may use connectors like: AND, OR and NOT.

These relational can use as relational operators like  $=, \neq, \geq, <, >, \leq$ .

Example- Consider following LOAN Relation,

| BRANCH_NAME | LOAN_NO | AMOUNT |
|-------------|---------|--------|
| Downtown    | L-17    | 1000   |
| Redwood     | L-23    | 2000   |
| Perryride   | L-15    | 1500   |
| Downtown    | L-14    | 1500   |
| Mianus      | L-13    | 500    |
| Roundhill   | L-11    | 900    |
| Perryride   | L-16    | 1300   |

**Input:**  $\sigma \text{ BRANCH\_NAME} = \text{"perryride"} (\text{LOAN})$

**Output:**

| BRANCH_NAME | LOAN_NO | AMOUNT |
|-------------|---------|--------|
| Perryride   | L-15    | 1500   |
| Perryride   | L-16    | 1300   |

**2) Project Operation:**

- This operation shows the list of those attributes that we wish to appear in the result. Rest of the attributes are eliminated from the table.
- It is denoted by  $\Pi$

Notation:  $\Pi A_1, A_2, \dots, A_n (r)$

Where,

$A_1, A_2, \dots, A_n$  is used as an attribute name of relation  $r$ .

Example: consider following CUSTOMER relation,

| NAME    | STREET  | CITY     |
|---------|---------|----------|
| Jones   | Main    | Harrison |
| Smith   | North   | Rye      |
| Hays    | Main    | Harrison |
| Curry   | North   | Rye      |
| Johnson | Alma    | Brooklyn |
| Brooks  | Senator | Brooklyn |

**Input:**

$\Pi$  NAME, CITY (CUSTOMER)

**Output:**

| NAME    | CITY     |
|---------|----------|
| Jones   | Harrison |
| Smith   | Rye      |
| Hays    | Harrison |
| Curry   | Rye      |
| Johnson | Brooklyn |
| Brooks  | Brooklyn |

**3) Union Operation:**

- Suppose there are two relations  $R$  and  $S$ . The union operation contains all the tuples that are either in  $R$  or  $S$  or both in  $R \& S$ .
- It eliminates the duplicate tuples.
- It is denoted by  $\cup$ .

Notation:  $R \cup S$

- A union operation must hold the following condition:
  - $R$  and  $S$  must have the attribute of the same number.
  - Duplicate tuples are eliminated automatically.

Example: Consider, following BORROW and DEPOSITOR relations-

**BORROW**

| CUSTOMER_NAME | LOAN_NO |
|---------------|---------|
| Jones         | L-17    |
| Smith         | L-23    |
| Hayes         | L-15    |
| Jackson       | L-14    |
| Curry         | L-93    |
| Smith         | L-11    |
| Williams      | L-17    |

**DEPOSITOR**

| CUSTOMER_NAME | ACCOUNT_NO |
|---------------|------------|
| Johnson       | A-101      |
| Smith         | A-121      |
| Mayes         | A-321      |
| Turner        | A-176      |
| Johnson       | A-273      |
| Jones         | A-472      |
| Lindsay       | A-284      |

**Input:**

$\sqcup$  CUSTOMER\_NAME (BORROW)  $\cup$   $\sqcup$  CUSTOMER\_NAME (DEPOSITOR)

**Output:**

| CUSTOMER_NAME |
|---------------|
| Johnson       |
| Smith         |
| Hayes         |
| Turner        |
| Jones         |
| Lindsay       |
| Jackson       |
| Curry         |
| Williams      |
| Mayes         |

**4) Set Intersection:**

- Suppose there are two relations R and S. The set intersection operation contains all tuples that are in both R & S i.e it finds common tuples.
- It is denoted by intersection  $\cap$ .

Notation:  $R \cap S$

Example: Using the above DEPOSITOR table and BORROW table-

**Input:**

$\sqcup$  CUSTOMER\_NAME (BORROW)  $\cap$   $\sqcup$  CUSTOMER\_NAME (DEPOSITOR)

**Output:**

| CUSTOMER_NAME |
|---------------|
| Smith         |
| Jones         |

**5) Set Difference:**

- Suppose there are two relations R and S. The set difference operation contains all tuples that are in R which are not in S.
- It is denoted by intersection minus (-).

Notation:  $R - S$

Example: Using the above DEPOSITOR table and BORROW table

**Input:**

$\sqcup$  CUSTOMER\_NAME (**BORROW**) -  $\sqcup$  CUSTOMER\_NAME (**DEPOSITOR**)

**Output:**

| CUSTOMER_NAME |
|---------------|
| Hayes         |
| Jackson       |
| Curry         |
| Williams      |

**6) Cartesian product:**

- The Cartesian product is used to combine each row in one table with each row in the other table.
- It is also known as a cross product.
- It is denoted by  $\times$ .

Notation:  $E \times D$

Example: Consider following EMPLOYEE and DEPARTMENT relations-

**DEPARTMENT**

## EMPLOYEE

| EMP_ID | EMP_NAME | EMP_DEPT |
|--------|----------|----------|
| 1      | Smith    | A        |
| 2      | Harry    | C        |
| 3      | John     | B        |

| DEPT_NO | DEPT_NAME |
|---------|-----------|
| A       | Marketing |
| B       | Sales     |
| C       | Legal     |

Input:

EMPLOYEE x DEPARTMENT

Output:

| EMP_ID | EMP_NAME | EMP_DEPT | DEPT_NO | DEPT_NAME |
|--------|----------|----------|---------|-----------|
| 1      | Smith    | A        | A       | Marketing |
| 1      | Smith    | A        | B       | Sales     |
| 1      | Smith    | A        | C       | Legal     |
| 2      | Harry    | C        | A       | Marketing |
| 2      | Harry    | C        | B       | Sales     |
| 2      | Harry    | C        | C       | Legal     |
| 3      | John     | B        | A       | Marketing |
| 3      | John     | B        | B       | Sales     |
| 3      | John     | B        | C       | Legal     |

## Theory Assignment

### A) Short Answer Questions (2–3 marks each)

1. Define a database.
2. What is data redundancy?
3. Write any two disadvantages of a traditional file system.
4. What is the role of a Database Administrator (DBA)?
5. Name any three examples of DBMS software.
6. Define metadata with an example.
7. What is data atomicity?
8. Who are naive users?
9. State any two differences between DBMS and a file system.
10. Define data integrity.
11. Define the role of a Database Administrator (DBA).
12. What is meant by database schema?
13. Differentiate between schema and instance.
14. List any four common data types used in databases.
15. Explain the concept of Physical Data Independence.
16. What is the purpose of Data Definition Language (DDL)? Give examples.
17. Write any two differences between DBMS and RDBMS.
18. Define Primary Key and Foreign Key with examples.
19. What is normalization? Why is it needed?
20. Mention any two disadvantages of Hierarchical data model.

---

### B) Long Answer Questions (5–8 marks each)

1. Explain the disadvantages of a traditional file processing system with examples.
2. Describe the advantages of DBMS in detail.
3. Explain the components of a DBMS with a neat diagram.
4. Discuss different types of database users with examples.
5. Explain in detail the roles and responsibilities of a DBA.
6. Describe the concept of data independence and its types.
7. Write a detailed note on concurrency control in DBMS.
8. Compare DBMS and file systems in a tabular form.
9. Explain backup and recovery mechanisms in DBMS.
10. What is data security? How is it achieved in DBMS?
11. Explain the major responsibilities of a DBA.
12. Describe 1-tier, 2-tier, and 3-tier DBMS architectures with suitable examples.
13. Discuss the three levels of DBMS architecture (External, Conceptual, Internal).
14. Explain different types of database languages: DDL, DML, DCL, TCL with examples.

15. Define Data Independence. Differentiate between Logical and Physical Data Independence with examples.
  16. Describe the different data models (Hierarchical, Network, ER, Relational, Hybrid) with advantages and disadvantages.
  17. Explain the components of an ER diagram with suitable examples.
  18. What are the types of relationships in ER model? Illustrate each with an example.
  19. Discuss Generalization, Specialization, and Aggregation in ER model.
  20. Explain Codd's 12 rules of RDBMS in detail.
  21. Define and explain the steps of Normalization up to 3NF/BCNF with example.
  22. What are relational algebra operations? Explain Select, Project, Union, and Cartesian product with examples.
- 

### C) Multiple Choice Questions (MCQs)

1. Which of the following is an example of DBMS software?  
a) MS Word      b) Oracle    c) Photoshop    d) Notepad  
**Answer:** b) Oracle
2. Which concept ensures that a database transaction is either fully completed or not executed at all?  
a) Data Security    b) Data Integrity    c) Data Atomicity      d) Data Redundancy  
**Answer:** c) Data Atomicity
3. In a traditional file system, duplication of data is known as:  
a) Data Isolation    b) Data Redundancy      c) Data Inconsistency    d) Data Dependence  
**Answer:** b) Data Redundancy
4. Who designs the structure of a database including tables and constraints?  
a) DBA              b) Database Designer    c) System Analyst      d) Casual User  
**Answer:** b) Database Designer
5. Which of the following is NOT a disadvantage of DBMS?  
a) Costly            b) Complexity      c) Automatic Backup    d) Loss of Data  
**Answer:** c) Automatic Backup
6. DBMS ensures data correctness using:  
a) Constraints      b) Firewalls          c) Metadata      d) Indexes  
**Answer:** a) Constraints
7. In DBMS, separation of data from its application program is called:  
a) Data Redundancy    b) Data Independence      c) Data Integrity      d) Data Isolation  
**Answer:** b) Data Independence
8. Which user type frequently uses pre-developed applications but has no DBMS knowledge?  
a) Casual User    b) System Analyst      c) Parametric End User      d) Sophisticated User  
**Answer:** c) Parametric End User
9. Which of these is NOT a component of DBMS?  
a) Hardware      b) Procedures      c) Software      d) Firmware  
**Answer:** d) Firmware
10. The database user who occasionally accesses the database but needs new information each time is called:  
a) Application Programmer    b) Casual User      c) DBA          d) Database Designer  
**Answer:** b) Casual User
11. Which of the following is not a responsibility of a DBA?  
a) Taking backups  
b) Monitoring performance  
c) Writing user application programs  
d) Allocating storage  
**Answer:** c)
12. Which of the following is an example of DML command?  
a) CREATE  
b) ALTER  
c) SELECT  
d) GRANT  
**Answer:** c)
13. In a 3-tier architecture, the client communicates with:  
a) Database directly  
b) Application server

- c) File system
- d) Data dictionary

**Answer: b)**

14. Which type of schema describes the physical storage of data?

- a) Logical Schema
- b) View Schema
- c) Physical Schema
- d) Conceptual Schema

**Answer: c)**

15. Which key uniquely identifies a record in a table?

- a) Foreign Key
- b) Primary Key
- c) Candidate Key
- d) Composite Key

**Answer: b)**

16. Which normal form removes partial dependency?

- a) 1NF
- b) 2NF
- c) 3NF
- d) BCNF

**Answer: b)**

17. Relational algebra operation used to remove unwanted columns is:

- a) Select ( $\sigma$ )
- b) Project ( $\Pi$ )
- c) Union ( $\cup$ )
- d) Intersection ( $\cap$ )

**Answer: b)**

18. Which of the following is an example of RDBMS?

- a) XML Database
- b) MySQL
- c) File System
- d) Spreadsheet

**Answer: b)**

19. In ER diagram, a multivalued attribute is represented by:

- a) Double rectangle
- b) Dashed oval
- c) Double oval
- d) Diamond

**Answer: c)**

20. Codd's Rule 2 (Guaranteed Access Rule) states that:

- a) All data must be accessible using table name, primary key, and column name
- b) All NULL values must be handled uniformly
- c) Data must be stored in tables only
- d) Database must support SQL

**Answer: a)**